

Political Bias Classification of Congressional Tweets Using Classical Machine Learning Models

Velkur Tanisha Reddy - SRN: PES2UG24AM181

Gagan K C - SRN: PES2UG24AM201

ABSTRACT

This paper compares three classical models, Bernoulli Naive Bayes, Multinomial Naive Bayes and a linear SVM, on the task of predicting whether a US Congress tweet was posted by a Democrat or a Republican account. Inspired by the Stanford CS229 study on political bias in news, the experiment uses a cleaned and de-duplicated Kaggle tweet dataset of 84,455 tweets with a reproducible stratified 80/10/10 split. All three models reach about 80% accuracy, and Multinomial Naive Bayes is selected on validation macro F1, achieving 79.94% accuracy and 0.7988 macro F1 on the held-out test set. The gap between models is too small to declare a clear winner.

***Index Terms*—political bias, text classification, Naive Bayes, support vector machine, TF-IDF, tweets**

I. INTRODUCTION

This work starts from the Stanford CS229 report on categorizing news articles by political bias [1]. The aim is to measure how well simple, well-known models handle a similar task, framed here as binary text classification: given a piece of political text, predict whether it came from a Democrat or a Republican. Following [1], three models are trained and compared under an identical setup: Bernoulli Naive Bayes, Multinomial Naive Bayes and a linear SVM.

An important caveat applies. The label is the party of the account that posted the text, not an independently verified bias score. The task therefore predicts party, which is related to political bias but not the same thing.

II. DATASET

The Democrat vs. Republican Tweets dataset from Kaggle is used [2], specifically the file `ExtractedTweets.csv`, which contains tweets from members of the US Congress together with the handle and party of each account. The raw file has 86,460 rows (44,392 Republican and 42,068 Democrat), so the classes are fairly balanced. Democrat is encoded as 0 and Republican as 1.

After cleaning, 86,293 rows remained. Repeated tweet texts were then removed across the whole dataset, leaving 84,455 unique tweets. These were split 80/10/10, stratified by party, with `random_state=42` for reproducibility:

- Training set: 67,564 tweets
- Validation set: 8,445 tweets
- Test set: 8,446 tweets

III. METHODOLOGY

A. Data Cleaning and Leakage Control

Rows with missing values were dropped, only the two valid party labels were kept, and tweets whose exact text appeared under both parties were removed because they add only label noise. Exact duplicates on tweet, party and handle were also dropped. Duplicate texts were removed before splitting, since retweets and copy-pasted messages are common and would otherwise let the same tweet appear in both train and test, inflating the score.

B. Text Variants

Three versions of the text were prepared. Variant A only lowercases and normalizes whitespace. Variant B removes URLs, converts punctuation to spaces and keeps the words inside hashtags and mentions. Variant C removes URLs, mentions and hashtags entirely. Models were trained on Variant B.

C. Models and Features

Each model received the input format it expects: binary counts for Bernoulli NB, raw counts for Multinomial NB and TF-IDF for LinearSVC. Every model was fitted on the training set only and compared on the validation set using macro F1. The test set was used once, at the end, for the selected model.

D. Implementation

The implementation uses Python with pandas, NumPy and scikit-learn [3]. One notebook performs loading, cleaning, variant creation, the stratified split and saving of the splits. A second notebook builds one scikit-learn pipeline per model, with the vectorizer first and the classifier second, so the vocabulary is learned from training data only. A helper function fits each pipeline and reports accuracy with macro precision, recall and F1 on the validation set. The selected model is scored on the test set and saved with joblib. All models use default settings, and LinearSVC uses `random_state=42`. Code is available in [4].

IV. RESULTS

Table I gives validation performance. Multinomial NB obtained the highest validation macro F1 and was selected.

TABLE I. VALIDATION PERFORMANCE OF THE THREE MODELS

Model	Representation	Val. Acc. (%)	Val. Macro F1
Bernoulli NB	Binary counts	79.96	0.7989
Multinomial NB	Counts	79.98	0.7992
LinearSVC	TF-IDF	79.89	0.7986

On the test set, Multinomial NB reached 79.94% accuracy, 80.01% macro precision, 79.86% macro recall and a macro F1 of 0.7988. These values are very close to the validation numbers. Because the model was chosen on validation and the test set was used only once, the test result is a fair estimate of performance on this dataset.

V. DISCUSSION AND CONCLUSION

All three models reach around 80%, and the gap between best and worst is only about 0.1 point of macro F1. This is too small to call a real winner, so Multinomial NB ranking first is closer to a tie than a finding. A fast Naive Bayes model with default settings performs as well as a linear SVM, which suggests most of the signal lies in which words are used. Exceeding 80% would likely require richer features such as bigrams or word embeddings.

Several limitations apply. The labels reflect party rather than verified bias, the tweets come from one set of accounts over one period, and tweets are short and informal. The same accuracy should not be expected on long news articles such as those in [1]. This is a related experiment, not an exact reproduction of that study.

Future Work

Future work includes comparing text variants A and C with B, tuning smoothing and regularization parameters with cross-validation, and evaluating the same models on real news articles.

REFERENCES

- [1] Stanford CS229 course project report on categorizing news articles by political bias (Stanford University).
- [2] “Democrat vs. Republican Tweets,” Kaggle dataset, file `ExtractedTweets.csv`. [Online]. Available: <https://www.kaggle.com>
- [3] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [4] V. T. Reddy and G. K. C., “political-bias-classification,” GitHub repository. [Online]. Available: <https://github.com/tanishareddy/political-bias-classification>